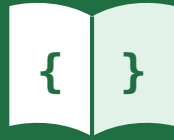

Modül 16 — Async, Zamanlama ve Spring Batch

Ödevler

Spring Boot Edu



2026-09-24

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — En Ucuz Kargo Firması (Kolay)	2
Ödev 2 — Bozuk Satırları Atlayan Bir Import (Orta)	3
Ödev 3 — Gecelik Rapor Job'ı (Orta)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/16-async-scheduling-batch/exercise/` altındadır. TODO yorumlarını bulun.
2. Ödevler Docker istemez: Veritabanı olmadan Spring Batch 6 job durumunu bellekte tutar.
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/16-async-scheduling-batch/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/16-async-scheduling-batch/solution/` altındaki çözümü okuyun.

Ödev 1 — En Ucuz Kargo Firması (Kolay)

Hedef: Üç kargo firması gönderim fiyatı sunar. Her yanıt yaklaşık 300 ms sürer. Hepsine **paralel** sorun ve bir firma başarısız olsa bile en ucuz teklifi alın.

`CarrierClient.quote(carrier, isbn)` hazır verilmiştir ve zaten `@Async`'tir.

Yapılacaklar (`exercisel.ShippingQuotes.cheapest`):

- TODO 1a — `CarrierClient.CARRIERS` içindeki her firmaya sorun. Herhangi birini beklemeden önce tüm çağrılar başlatın.
- TODO 1b — Başarısız olan bir firma teklif vermez. Bu, aramanın tamamını başarısız yapmamalıdır.
- TODO 1c — En düşük fiyatlı teklifi döndürün.

İpuçları:

- Ders bölüm 3.1: Önce `map(carrier -> carriers.quote(carrier, isbn))` ile bir listeye toplayın, sonra her future için `join()` çağırın.
- `future.exceptionally(error -> null)`, başarısız bir future'ı null ile tamamlanan bir future'a çevirir.
- `min(Comparator.comparing(Quote::price))`.

Kabul kriterleri: `ExerciselTest` içindeki 3 testin hepsi geçer (paralel olanı 700 ms'nin altında bitmelidir).

Tahmini süre: 20 dakika

Ödev 2 — Bozuk Satırları Atlayan Bir Import (Orta)

Hedef: Bir sipariş dosyasında birkaç bozuk satır vardır (ISBN yok, miktar “zero”). Bunlar atlanmalı ve iyi satırlar içe aktarılmalıdır. Ama çoğu bozuk olan bir dosya muhtemelen yanlış dosyadır: O zaman job başarısız olmalıdır.

Reader, processor (`InvalidOrderLineException` fırlatır) ve writer (`OrderInbox`) hazır verilmiştir.

Yapılacaklar (`exercise2.ImportOrdersJobConfiguration`):

- TODO 2a — Step’i hataya dayanıklı yapın: `InvalidOrderLineException` fırlatan satırları atlayın ...
- TODO 2b — ... ama en fazla 3 tanesini. Ondan sonra job başarısız olur.

İpuçları:

- Ders bölüm 3.4: Writer’dan sonra `.faultTolerant().skip(...).skipLimit(...)`.
- `orders.csv` 2 bozuk satır içerir (→ `COMPLETED`, 5 sipariş), `orders-mostly-broken.csv` 4 tane içerir (→ `FAILED`).
- Bu projede iki job vardır. Bu yüzden testler `jobs.setJob(...)` ile birini seçer.

Kabul kriterleri: `Exercise2Test` içindeki iki test de geçer.

Tahmini süre: 20 dakika

Ödev 3 — Gecelik Rapor Job’ı (Orta)

Hedef: Her gece 02:00’de önceki günün satışlarının raporu oluşturulur. İş, `report.date` job parametrelili bir batch job’dır. Böylece herhangi bir gün elle yeniden raporlanabilir. Zamanlayıcı yalnızca job’ı başlatır.

Yapılacaklar (`exercise3` paketi):

- TODO 3a — `DailySalesReportJobConfiguration`: Tasklet date gününün satışlarını sayar, tutarlarını toplar ve arşive bir `DailyReport` kaydeder.
- TODO 3b — `NightlyReportScheduler.createReport`: Her gece İstanbul saatiyle 02:00’de çalışsın.
- TODO 3c — `dailySalesReportJob`’ı saate göre `report.date = dün job parametresiyle` başlatın.

İpuçları:

- Tasklet tarihi zaten alır: `@Value("#{jobParameters['report.date']}") LocalDate date`.
- `new JobParametersBuilder().addLocalDate("report.date", date).toJobParameters()`, ardından `jobs.start(job, parameters)`.

- Enjekte edilen Clock'u kullanın: `LocalDate.now(clock).minusDays(1)`. Test saati 25 Eylül 02:00'ye ayarlar, bu yüzden rapor 24 Eylül için olmalıdır.
- Ders bölüm 3.2: `@Scheduled(cron = "0 0 2 * * *", zone = "Europe/Istanbul")`.

Kabul kriterleri: Exercise3Test içindeki iki test de geçer.

Tahmini süre: 30 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Ders uygulamasını iki kez başlatın (iki terminal, ikincisi için `--server.port=8081`). `bo-okstore.report.cron`'u raporun her dakika çalışacağı şekilde değiştirin ve **iki** örnekte de çalıştığını izleyin. Ardından onu ShedLock ile koruyun (`shedlock-spring` ve `shedlock-provider-jdbc-template`, kilit tablosu PostgreSQL'de) ve dakikada yalnızca bir kez çalıştığını kontrol edin.